

On the use of Pareto-based Features in Meta-Learning for Multi-Objective Offshore Wind Farm Layout Optimization

Gustavo J. N. Silva¹[0009–0003–1559–5502], João G. L. S. G.
Silva¹[0009–0005–9075–0124], and Islame F. C. Fernandes¹[0000–0003–3534–8042]

Federal University of Bahia, Institute of Computing
{gustavonovaes, joao.lofiego, islame.felipe}@ufba.br

Abstract. The Multi-objective Wind Farm Layout Optimization Problem (*MoWFLOP*) seeks the optimal turbine placement in offshore wind farms, where construct cost minimization and power generation maximization are simultaneously optimized. As *MoWFLOP* belongs to *NP-hard*, previous work has focused on metaheuristic approaches to compute high-quality solutions. However, metaheuristics’ results can vary across wind farm instances, making manual algorithm selection impractical. This paper investigates the use of Pareto-based meta-features to guide a landscape-aware method based on meta-learning for automated metaheuristic selection applied to *MoWFLOP*. To our knowledge, no such investigation exists for this problem. We explore 30 different sampling configurations to extract Pareto-based meta-features from 300 wind farms, balancing computational cost with feature informativeness. We build a dataset for each configuration and train Random Forest models to predict the best-performing algorithm for unseen instances. The analyses comprised the metaheuristics’ results regarding Pareto-compliant indicators, the impact of sampling configurations on feature values, and the meta-learning performance for automated metaheuristic selection.

Keywords: Wind Farm Layout · Multi-objective optimization · Meta-learning

1 Introduction

Wind energy has experienced remarkable growth in recent years, driven by the urgent need to address climate change. In particular, *offshore* wind farms have become a focus for investment, owing to the significantly higher and more consistent wind resources available in the high sea than onshore sites [14]. The design of offshore wind farms involves various optimization tasks [17].

This paper studies the *Multi-objective Offshore Wind Farm Layout Optimization Problem* (*MoWFLOP*), seeking the optimal placement of offshore turbines within a designated area to minimize construction costs and maximize energy production. In *MoWFLOP*, the power output of turbines relies not only on their positions but also on their spatial arrangement and the distances between them

[20]. A significant hurdle in this problem is the *wake effect* [1], which occurs when upwind turbines reduce wind speed for downwind turbines, thereby diminishing their energy efficiency. This phenomenon adds complexity to *MoWFLOP*, necessitating optimizing spatial layouts and considering interactions among turbines.

As *MoWFLOP* belongs to *NP-hard* [23], previous work has focused on developing efficient metaheuristics [3, 19, 21, 6, 23]. These algorithms have demonstrated the ability to generate high-quality solutions, emphasizing evolutionary algorithms that effectively balance exploration and exploitation of the search space. Nonetheless, the performance of metaheuristics is contingent upon the characteristics of the underlying landscape of the instance solved, which encompasses factors such as difficulty, ruggedness, and multi-modality [15]. So, selecting the most appropriate metaheuristic for a new wind farm design is not trivial.

This study investigates a meta-learning pipeline for automated metaheuristic selection in the *MoWFLOP* context, leveraging *Fitness Landscape Analysis* (*FLA*) techniques. *FLA* quantifies problem instance characteristics by extracting landscape *meta-features*, which serve to predict algorithm performance [15]. We derive Pareto-based meta-features from sampled solutions to characterize each instance’s search space. These meta-features are used to build a *meta-dataset*, where instances are labeled with their best-performing algorithms. A machine learning model is then trained on this meta-dataset to address the *Algorithm Selection Problem* (*ASP*) [18], recommending the best metaheuristics for unseen instances. Besides facilitating automated algorithm selection, this data-driven approach enables effective hybridization strategies [12, 11, 10]. To our knowledge, no prior work has applied this methodology to wind farm design.

This study employs two neighborhood-based methods (random walk and adaptive walk) to compute 20 Pareto-based meta-features [8]. It examines 30 distinct sampling configurations to assess the impact of meta-feature extraction computational efforts. We generated 290 new benchmark instances that simulate real-world wind farms, encompassing diverse difficulty levels. Additionally, we incorporated 10 instances from existing literature [6] and executed each configuration across these 300 instances. We created a meta-dataset for every configuration. Given their prevalence in the *MoWFLOP* literature [3, 19], we used *NSGA-II* [9] and *MOEA/D* [22] as the benchmark metaheuristics. A Random Forest model for each dataset predicts the best algorithm for unseen instances.

Pareto-compliant quality indicators and statistical tests measured the quality of the solutions produced by *NSGA-II* and *MOEA/D*. The analyses included the number of objective function evaluations needed for each sampling configuration and their impact on meta-feature informativeness. Additionally, we measured the effectiveness of meta-learning in metaheuristic selection applied to *MoWFLOP* and assessed various metrics in regression tasks, including MSE, MAE, RMSE, MAPE, and R^2 . Instances, source codes, datasets, and raw experimental and processed results are available at a GitHub Repository: https://github.com/wflopbracis/WFLOP_meta_learning.

The main contributions of this paper are (1) a meta-learning-based method for the automatic selection of metaheuristics applied to *MoWFLOP* and (2) a

cost-vs-accuracy study on Pareto-based meta-features to characterize the *MoWFLop* landscape. Additional contributions include new *MoWFLop* instances and an experimental analysis of *NSGA-II* and *MOEA/D* on such instances.

2 Related Work

Formalized by [18], *ASP* assumes that selecting the best algorithm for a problem requires understanding its strengths and weaknesses. This problem has inspired subsequent research in meta-learning [4], which utilizes knowledge from prior algorithms’ executions to forecast the most effective algorithm for previously unseen instances. Reference [4] systematized this process, emphasizing pipelines for meta-feature extraction, meta-dataset construction, and model training and testing. However, *ASP* becomes more challenging in multi-objective optimization due to the trade-offs between conflicting objectives.

FLA has been increasingly studied for multi-objective optimization [26]. Reference [8] differentiates between global landscape meta-features, which necessitate a complete enumeration of the search space, and local landscape meta-features, which derive from sampling the search space. Existing works on multi-objective optimization have focused on local meta-features. Liefvooghe et al. [15] proposed meta-features based on Pareto dominance and quality indicators, emphasizing that these meta-features correlate significantly with the effectiveness of evolutionary algorithms. Cosson et al. [7] introduced decomposition-based meta-features, analyzing scalarized subproblems’ landscapes to minimize dimensionality. Cosson et al. [8] further contrasted the cost and accuracy of these three meta-feature groups, reporting the impact of sampling strategy on the accuracy of automated algorithm selection.

Despite these advancements, several challenges persist. Most research has concentrated on theoretical multi-objective problems, resulting in limited applicability to real-world scenarios, such as *MoWFLop*. This gap underscores the motivation and justification for the present paper.

3 Background concepts

3.1 Algorithm Selection Problem

Let $\mathcal{I}$ denote a set of instances, $\mathcal{F} \subseteq \mathbb{R}^\eta$ a collection of η -tuple meta-features within the search space, and $h : \mathcal{I} \rightarrow \mathcal{F}$ a function that associates each instance with an η -tuple of meta-features. Let $\mathcal{A}$ be a set of algorithms, and $\mathcal{Y} \in \mathcal{R}$ a collection of performance indicators. The *ASP* seeks a function $\mathcal{S} : \mathcal{F} \rightarrow \mathcal{A}$ that maps the characteristics $h(\iota) \in \mathcal{F}$ of an instance $\iota \in \mathcal{I}$ to the most effective algorithm $\alpha \in \mathcal{A}$ for ι , based on the metrics specified in $\mathcal{Y}$ [18].

3.2 Multi-objective optimization

Multi-objective problems involve the simultaneous optimization of $q > 1$ objective functions. Let Γ be the set of feasible solutions, $\mathcal{Y} \subseteq \mathbb{R}^q$ the objective

space, and $f : \Gamma \rightarrow \mathcal{Y}$ a function mapping solution $\gamma \in \Gamma$ to an objective vector $f(\gamma) = (f_1(\gamma), \dots, f_q(\gamma)) \in \mathcal{Y}$. Expression (1) defines a multi-objective problem:

$$\text{optimize}_{\gamma \in \Gamma} (f_1(\gamma), \dots, f_q(\gamma)) \quad (1)$$

In this work, “optimization” refers to the Pareto dominance relation. A solution $\gamma \in \Gamma$ dominates another solution $\gamma' \in \Gamma$, denoted $\gamma \prec \gamma'$, if and only if $f_k(\gamma)$ is better than or equal to $f_k(\gamma')$ for all $k = 1, \dots, q$, and $f_k(\gamma)$ is better than $f_k(\gamma')$ for at least one k . The Pareto-optimal set is $\Gamma^* = \{\gamma \in \Gamma \mid \nexists \gamma' \in \Gamma \text{ such that } \gamma' \prec \gamma\}$. Set $\mathcal{Y}^* \subseteq \mathcal{Y}$ containing objective vectors $f(\gamma)$, $\gamma \in \Gamma^*$, is the Pareto-optimal front. Set $\mathcal{Y}^{*'} \subseteq \mathcal{Y}$ is a Pareto-optimal front approximation if and only if $\forall v, v' \in \mathcal{Y}^{*'}, \gamma = f(v)$ and $\gamma' = f(v')$, $\gamma \not\prec \gamma'$ and $\gamma' \not\prec \gamma$.

3.3 Multi-objective Offshore Wind Farm Layout Optimization

Let τ denote the total number of turbines, and let $\mathcal{P}$ represent the set of available locations for turbine installation in the open sea. For each $i \in \mathcal{P}$, a binary decision variable z_i indicates whether a turbine is to be constructed at location i ($z_i = 1$) or not ($z_i = 0$). The construction cost associated with installing a turbine at location i is represented by C_i [6].

Each turbine generates a specific amount of power from the unobstructed wind. However, the turbine’s rotor generates a region of reduced wind speed behind it, known as *wake effect*, which affects the power production of other turbines situated in that region [1]. This paper employs the method described by [13] to compute the wake effect. Let E be an $|\mathcal{P}| \times |\mathcal{P}|$ matrix, where $E_{ii} \geq 0$ is the amount of power production of the turbine at position $i \in \mathcal{P}$ from the free wind, and $E_{ij} \leq 0$ is the power losses incurred by the turbine at position i due to the wake effect caused by the turbine at position j , for all $j \in \mathcal{P}$, with $i \neq j$.

The *MoWFLOP* focuses on placing τ turbines across a set of available positions $\mathcal{P}$, simultaneously maximizing the power production and minimizing the construction cost. Expressions (2)-(5) presents the *MoWFLOP* formulation. Expression (2) is the objective function aimed at minimizing the construction expenses, whereas expression (3) targets the maximization of power generation. Constraint (4) ensures that the number of turbine placements matches the total number of turbines. Constraints (5) prevent the placement of turbines at positions i and j if they are closer than the minimum required distance σ , represented by the Euclidean distance d_{ij} , to ensure that the physical loads on adjacent turbines remain within their design limits.

$$\min \sum_{i \in \mathcal{P}} C_i z_i \quad (2)$$

$$\max \sum_{i \in \mathcal{P}} \sum_{j \in \mathcal{P}} E_{ij} z_i z_j \quad (3)$$

$$\sum_{i \in \mathcal{P}} z_i = \tau \quad (4)$$

$$z_i + z_j \leq 1 \quad \forall i, j \in \mathcal{P}, d_{ij} \leq \sigma \quad (5)$$

4 Meta-learning

In this paper, $\mathcal{I}$ represents a set of *MoWFLOP* instances, while $\mathcal{F}$ is a collection of numerical measures known as *meta-features*, which characterize the search space. The meta-feature extraction corresponds to defining $h : \mathcal{I} \rightarrow \mathcal{F}$. Additionally, $\mathcal{A}$ is a set of algorithms implemented for *MoWFLOP*. *Meta-label* is the best-performing algorithm for each instance $\iota \in \mathcal{I}$. The dataset composed of meta-features and meta-labels is referred to as *meta-dataset*. A machine learning technique applied to the meta-dataset generates a model that, when presented with an unseen *MoWFLOP* instance, selects the most appropriate algorithm.

This section details the meta-dataset construction. Section 4.1 presents Pareto-based meta-features to *MoWFLOP*. Section 4.2 details the meta-features extraction. Section 4.3 describes the meta-labels.

4.1 Pareto-based meta-features

This work uses the Pareto-based meta-features proposed by [15, 8]. Let $\mathcal{N} : \Gamma \rightarrow 2^\Gamma$ define a neighborhood relation, where for *MoWFLOP* $\mathcal{N}(\gamma)$ represents the set of solutions differing from γ by the position of exactly one turbine. Consider an ordered sequence of solutions $\mathcal{W}_L = (\gamma^0, \gamma^1, \dots, \gamma^L)$, termed a *walk*, such that $\gamma^{t+1} \in \mathcal{N}(\gamma^t)$ for all $t = 0, \dots, L - 1$. Section 4.2 details methods for generating $\mathcal{W}_L$.

Pareto-based meta-features are measures based on Pareto dominance relations. For each solution γ^t in the walk, we compute four dominance-based measures: the proportion of solutions in $\mathcal{N}(\gamma^t)$ dominated by γ^t (*inf*), the proportion dominating γ^t (*sup*); the proportion incomparable to γ^t (*inc*); the proportion of locally non-dominated solutions in $\mathcal{N}(\gamma^t)$ (*ind*). These measures are aggregated across $\mathcal{W}_L$ using five statistical functions: average (*avg*), standard deviation (*sd*), first auto-correlation coefficient (*r1*), kurtosis (*kr*), and skewness (*sk*). Each meta-feature is denoted as *c.s*, where $c \in \{\text{inf}, \text{sup}, \text{inc}, \text{ind}\}$ and $s \in \{\text{avg}, \text{sd}, \text{r1}, \text{kr}, \text{sk}\}$, yielding 20 distinct meta-features.

4.2 Meta-features extraction

Meta-feature extraction is performed by computing a walk $\mathcal{W}_L$ using *random walk* and *adaptive walk* methods [8]. Both methods start with an initial solution $\gamma^{t=0}$ and, during each iteration $t = 0, \dots, L - 1$, they select a solution γ^{t+1} from the neighbors $\mathcal{N}(\gamma^t)$ of γ^t . The key distinction between them lies in the selection strategy for γ^{t+1} . The random walk chooses γ^{t+1} randomly with uniform probability, whereas the adaptive walk selects γ^{t+1} from those solutions in $\mathcal{N}(\gamma^t)$ that dominate γ^t . In the case of the random walk, the walk length $L > 0$ is predetermined by the user. Conversely, the adaptive walk iterates until there are no solutions in $\mathcal{N}(\gamma^t)$ that dominate γ^t , making the length L dependent on the number of iterations necessary to reach a Pareto local optimal solution γ^L .

Random walk and adaptive walk use a truncated neighborhood, as suggested by [8]. Rate $r \in (0, 1]$ stands for the proportion of neighborhood explored for

Algorithm 1: Random walk

Input : $L > 0$: walk length
 $r \in (0, 1]$: proportion of explored neighborhood

```

1  $\gamma^0 \leftarrow$  a random solution
2  $t \leftarrow 0$ 
3 repeat
4    $\gamma^{t+1} \leftarrow$  a random solution from  $\mathcal{N}_r(\gamma^t)$ 
5    $t \leftarrow t + 1$ 
6 until  $t = L$ ;
```

Algorithm 2: Adaptive walk

Input : $r \in (0, 1]$: proportion of explored neighborhood

```

1  $\gamma^0 \leftarrow$  a random solution
2  $t \leftarrow 0$ 
3 repeat
4    $\mathcal{N}^* = \{\gamma' \in \mathcal{N}_r(\gamma^t) \mid \gamma' \prec \gamma^t\}$ 
5   if  $\mathcal{N}^* \neq \emptyset$  then
6      $\gamma^{t+1} \leftarrow$  a random solution from  $\mathcal{N}^*$ 
7      $t \leftarrow t + 1$ 
8   end
9 until  $\mathcal{N}^* = \emptyset$ ;
```

each solution. We compute a subset $\mathcal{N}_r(\gamma^t)$ of $r \times |\mathcal{N}(\gamma^t)|$ solutions sampled uniformly at random from $\mathcal{N}(\gamma^t)$, for all $t = 0, \dots, L - 1$. So, for $r = 1$ the full original neighborhood is explored, whereas for $r = 0.1$ means that 10% of neighbors are explored.

Two parameters control the random walk: the length $L > 0$ and rate $r \in (0, 1]$. Algorithm 1 outlines the random walk procedure. Line 1 initializes a random solution γ^0 , and line 2 sets counter t to 0. The loop between lines 3 and 6 iterates until $t = L$. In every iteration, line 4 randomly selects solution γ^{t+1} from $\mathcal{N}_r(\gamma^t)$. Algorithm 2 details the adaptive walk, where only rate r is provided as input. Line 4 computes the set $\mathcal{N}^* \subseteq \mathcal{N}_r(\gamma^t)$ of solutions that dominate γ^t . Line 6 selects γ^{t+1} randomly from $\mathcal{N}^*$. The procedure terminates when $\mathcal{N}^* = \emptyset$.

The complexity of meta-feature extraction belongs to $\Theta(L \cdot r)$, where varying combinations of parameters L and r may yield different meta-feature values, as noted by [8]. This study investigates the influence of these parameters on both the resulting meta-feature values and the meta-learning framework for *MoWFLOP*.

4.3 Meta-labels

MOEA/D [22] and *NSGA-II* [9] are the meta-label used in this study. They maintain a population P comprising N solutions, where N is defined by the user. The difference between them lies in their strategies to evolve the population throughout the evolutionary process. *MOEA/D* decomposes the problem into multiple sub-problems and addresses them concurrently, while *NSGA-II* employs a Pareto dominance-based approach.

MOEA/D maintains a set of N weight vectors $\Lambda \subseteq \mathbb{R}^q$, where q is the number of objective functions. Each vector $\lambda^j = (\lambda_1^j, \dots, \lambda_q^j) \in \Lambda$, for $j = 1, \dots, N$,

defines the j -th sub-problem, and the best-found solution for this sub-problem is the j -th individual in the population, referred to as γ^j . The goal of the j -th sub-problem is to minimize the Tchebycheff function, utilizing λ^j as the corresponding weight vector. Additionally, *MOEA/D* maintains a set $\aleph(j)$ comprising $\#ngb$ nearest weight vectors to λ^j , determined by Euclidean distance, with $\#ngb$ being a user-defined parameter. In every iteration for $j = 1, \dots, N$, *MOEA/D* randomly selects two indices, j' and j'' , from $\aleph(j)$ and employs crossover and mutation operators to produce an offspring solution $\bar{\gamma}$ derived from $\gamma^{j'}$ and $\gamma^{j''}$. For all $j''' \in \aleph(j)$, the algorithm replaces $\gamma^{j'''}$ with $\bar{\gamma}$ if $\bar{\gamma}$ demonstrates improved performance compared to $\gamma^{j'''}$ with respect to the Tchebycheff function for the j -th sub-problem.

In each iteration, *NSGA-II* generates an offspring population $\bar{P}$ of size N using crossover and mutation operators. Solutions in $P \cup \bar{P}$ are then ranked into distinct non-domination fronts. The first front, F_1 , comprises the non-dominated solutions from $P \cup \bar{P}$. The second front, F_2 , includes the non-dominated solutions from $P \cup \bar{P} - F_1$, and this ranking process continues accordingly. To form the population for the next iteration, N individuals are selected from F_1 , F_2 , and so on. If adding the k -th front (F_k) leads to exceeding N individuals, *NSGA-II* completes the population by selecting solutions from F_k based on the largest crowding distance among their nearest neighbors.

In this study, *NSGA-II* and *MOEA/D* use the same solution encoding, initial population, crossover, and mutation. The solution encoding consists of a $|\mathcal{P}|$ -sized numerical vector, where each entry either contains a turbine ID (if that position is occupied) or -1 (otherwise). The initial population is generated randomly with uniform probability. The crossover process receives two parent solutions and outputs two offspring solutions through a single-point split: the first child inherits the left segment from the first parent and incorporates the non-redundant turbines from the second parent (and vice versa for the second child). Mutation operates by randomly selecting a turbine and relocating it to a randomly chosen empty position.

5 Experimental methodology

5.1 *MoWFLOP* instances

The set $\mathcal{I}$ contains 300 synthetic instances, identified by an *ID* ranging from 1 to 300. The set includes 10 instances provided by [6] and 290 newly generated instances. Instance generation used the method and parameters outlined by [6]. Each instance has 1, 2, or 3 zones, each offering between 3238 and 21709 available positions and 0, 1, 2, or 3 physical obstacles (designated areas where turbines cannot be installed). The model presented in [2] calculated the foundation cost for turbine placement in available positions, considering water depths ranging from 10 to 40 meters. Each instance includes between 5 and 75 fixed turbines, representing nearby wind farms, which are not subject to optimization but may induce wake effects. The number of turbines to be positioned in the available positions varies from 1 to 100 per zone. The fixed turbines have a capacity of

10 MW, while the turbines to be installed are 15 MW. We employed the same power, thrust curve, and wind scenarios discussed by [6]. Details for each instance are in the GitHub repository.

The generation of new instances, representing a significant contribution of this paper, is driven by the necessity to develop a sufficiently large dataset to enable meta-learning tasks. These instances incorporate realistic features of wind farms such as boundaries, obstacles, and nearby installations. This allows for assessing techniques in scenarios that closely reflect practical conditions encountered in the industry [6].

5.2 Multi-objective test and assessment

The results for each instance concern 20 independent executions of *NSGA-II* and *MOEA/D*. The stopping criterion was 10^6 evaluations of the objective function for each algorithm. The parameters for *NSGA-II* and *MOEA/D* come from references [19] and [3], respectively.

Hypervolume (*HV*) [24] and additive epsilon (ϵ) [25] evaluated the quality of the approximation sets. These metrics require a reference set R for each instance, which we obtained by filtering the non-dominated vectors from the combined approximation sets produced by *NSGA-II* and *MOEA/D*. The *HV* quantifies the area of the objective space dominated by a set of non-dominated solutions. In this study, the *HV* results are the difference between the approximation set generated by the algorithm and the reference set R . We normalized each objective to the range $[1, 2]$ and utilized the point $(2.1, 0.9)$ to bound the objective space. The additive ϵ measures the minimal factor needed to transform each point in an approximation set so that the resulting set is weakly dominated by the reference set R . A lower value of both *HV* and ϵ indicates a superior approximation set. Mann–Whitney statistical test, with a significance level of 0.05, verified significant differences between results produced by *NSGA-II* and *MOEA/D*. Raw experimental and processed results, including mean, average, and standard deviation of *HV* and ϵ for each instance, are available at the GitHub repository.

5.3 Dataset construction

This study constructed 30 datasets using different random and adaptive walk sampling configurations. All datasets contain 300 instances and 20 meta-features described in section 4.1. The random walk has 25 different sampling configurations based on parameters $L \in \{5, 10, 25, 50, 100\}$ and $r \in \{0.05, 0.1, 0.25, 0.5, 1\}$. The adaptive walk considers five samples with $r \in \{0.05, 0.1, 0.25, 0.5, 1\}$. Because *NSGA-II* and *MOEA/D* presented more different performance profiles concerning ϵ in the instance set, we used ϵ to compute the meta-labels. For each instance, the algorithm with the smallest average ϵ over 20 independent executions is the meta-label of the corresponding instance. The datasets are available at the GitHub repository.

The analyses examined the variation of meta-feature values based on the parameters L and r [8]. For each combination of L and r , we assess the *GAP*

between the meta-feature value and the highest budget-demanding setting. Expression (6) computes this GAP for each instance and meta-feature ξ , where $V_{max}(\xi)$ represents the value of ξ in the highest budget-demanding scenario, and $V_{l,r}(\xi)$ denotes the value of ξ considering parameters L and r (or just r in the case of the adaptive walk). In the context of the random walk, $V_{max}(\xi)$ is achieved when $L = 100$ and $r = 1.0$, while for the adaptive walk $V_{max}(\xi)$ is associated with $r = 1.0$ and the maximum value of L achieved.

$$GAP = \frac{|V_{max}(\xi) - V_{l,r}(\xi)|}{V_{max}(\xi)} \quad (6)$$

5.4 Meta-learning test and assessment

Experiments followed the methodology in [8] to analyze the benefits of meta-learning and different sampling configurations to tackle the ASP for $MoWFLOP$. The instance set is divided into a training set $\mathcal{T}$ and a validation set $\mathcal{V}$. Set $\mathcal{T}$ is used to train a multi-output Random Forest (RF) regression model [5] to fit the ϵ value related to $NSGA-II$ and $MOEA/D$. Given the meta-features of an unseen instance $\iota \in \mathcal{V}$, the model predicts the ϵ value for each algorithm. The one with the smallest predicted ϵ is selected as the recommended algorithm. The Random Search method tuned the RF hyperparameters: $n_estimators = 1000$, $max_depth = 2$, $min_samples_split = 2$, and $min_samples_leaf = 3$.

The assessment considered the merit indicator, referred to as m^* , addressed by [8, 16]. This metric measures the distance between the predicted algorithm performance and two baselines: (i) the individual algorithm that performs best across all training instances, known as the *Single Best Solver* (SBS); and (ii) the performance of the *Virtual Best Solver* (VBS). Let $\overline{r\epsilon}(\alpha, \mathcal{J})$ be the average ϵ of algorithm $\alpha \in \mathcal{A}$ over a subset of instances $\mathcal{J} \subseteq \mathcal{I}$. SBS is the best algorithm in $\mathcal{T}$ achieved without meta-learning tasks, as in Expression (7). VBS selects the algorithm that performs the best in each instance $\iota \in \mathcal{I}$. Expression (8) computes m^* , where α_s is the algorithm predicted by RF for each instance $\iota \in \mathcal{V}$. A lower m^* indicates better configurations, where $m^* = 0$ indicates a perfect oracle, $m^* < 1$ (respectively $m^* > 1$) indicates that using RF performs better (respectively worse) than SBS .

Besides m^* , the prediction results were evaluated utilizing conventional regression metrics, namely MSE, MAE, RMSE, MAPE, and R^2 . A repeated random hold-out strategy was employed, implementing a 90/10% split for training and validation datasets [8]. The values are the average metric across 100 independent folds, ensuring a reliable assessment of the predictive performance.

$$SBS = \arg \min_{\alpha \in \mathcal{A}} \overline{r\epsilon}(\alpha, \mathcal{T}) \quad (7)$$

$$m^* = \frac{\overline{r\epsilon}(\alpha_s, \mathcal{V}) - \overline{r\epsilon}(VBS, \mathcal{V})}{\overline{r\epsilon}(SBS, \mathcal{V}) - \overline{r\epsilon}(VBS, \mathcal{V})} \quad (8)$$

6 Results and Discussion

6.1 Comparison between *NSGA-II* and *MOEA/D*

MOEA/D outperformed NSGA-II in terms of average ϵ across 236 instances, constituting 78.67% of the cases, while NSGA-II achieved the best average ϵ in 64 instances (21.33%). Regarding HV, MOEA/D recorded the best average in 290 instances (96.67%), whereas NSGA-II excelled in 10 (3.33%). The Mann–Whitney statistical test showed that both algorithms performed similarly across most instances, precisely in 81.67% and 100% of the cases, respectively, for HV and ϵ . Figure 1 illustrates the distribution of HV and ϵ values obtained by MOEA/D and NSGA-II on an instance (ID=105), where lower values signify better results. In this instance, the statistical test pointed out significant differences concerning the *HV*, while no differences were observed for ϵ results.

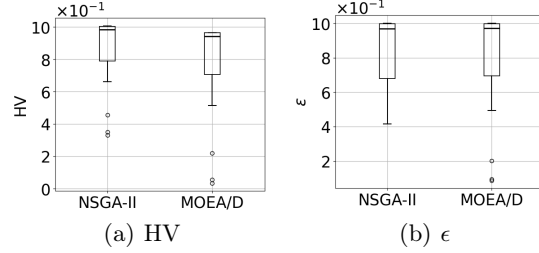


Fig. 1: Results for the instance ID=105

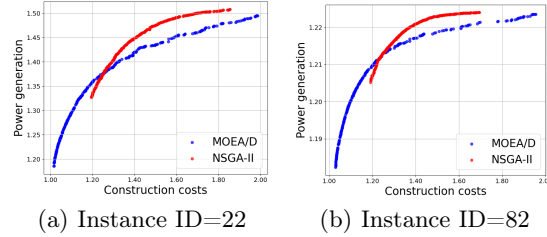


Fig. 2: Normalized Pareto front approximation

The strength of MOEA/D lies in its decomposition technique, which effectively balances diversity and convergence and allows MOEA/D to explore the objective space more thoroughly, reaching regions that NSGA-II does not access. In contrast, the Pareto dominance method utilized by NSGA-II limits its effectiveness in addressing the *MoWFLOP*, confining it to smaller regions within

the objective space. Figure 2 shows the Pareto front approximation obtained in a single run of two instances ($ID = 22$ and $ID = 82$). The x -axis represents construction costs, while the y -axis shows the power generated. Both objectives normalized within the range $[1, 2]$. The figure demonstrates that, although most solutions generated by NSGA-II dominate those by MOEA/D, the latter achieves a more dispersed distribution in the objective space, in contrast to the concentration seen with NSGA-II. This fact leads MOEA/D to have a larger dominance area. This trend is consistent across most instances.

6.2 Meta-features Analysis

As expected, random walk and adaptive walk methods increase the number of objective function evaluations with the value of r rises. The setting of $r = 1.0$ for both algorithms necessitated the highest number of evaluations. This configuration enables the algorithm to thoroughly explore the entire neighborhood of each solution along the walk, thereby elevating the number of objective function evaluations. Specifically, in the case of the random walk, with $L = 100$ and $r = 1.0$, the evaluations peaked.

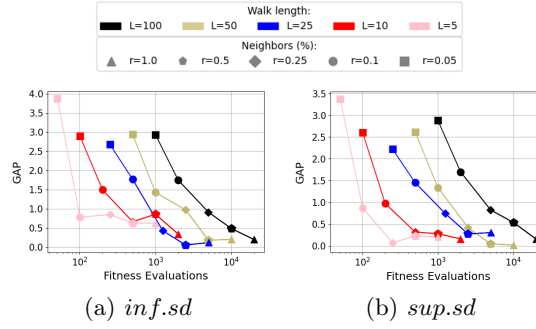


Fig. 3: Meta-feature relative deviation computed by the random walk

To illustrate the impact of objective function evaluations and parameters r and L on the meta-feature values produced by the random walk procedure, we selected two features: *inf.sd* and *sup.sd*. Figure 3 illustrates the normalized variation of *inf.sd* and *sup.sd* obtained through the random walk as a function of evaluation numbers for each setting of r and L [8]. The x-axis corresponds to the number of objective function evaluations, while the y-axis shows the *GAP*, calculated using the expression (6). Each colored line corresponds to a specific value of L , with five geometric markers indicating different values of r . Generally, the *GAP* tends to decrease as the value of r increases. The figure shows that the number of objective evaluations consumed by random walk increases with r and L . With $r = 1.0$, the values of $L \in \{25, 50, 100\}$ tend to lead the *GAP* of *inf.sd* to zero, whereas the *GAP* reaches approximately 5% and 2.5% at the settings of

$L = 5$ and $L = 10$, respectively. A similar analysis applies to *sup.sd*, where all settings of L achieve a *GAP* value between 0% and 5% at $r = 1.0$. These results suggest that higher values of r result in the random walk converging towards specific values for both *inf.sd* and *sup.sd*.

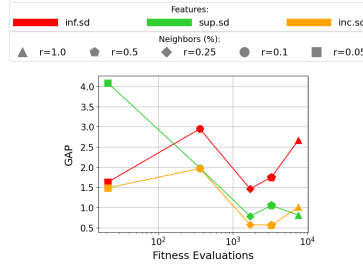


Fig. 4: Meta-feature relative deviation computed by the adaptive walk

Figure 4 illustrates the *GAP* achieved by the adaptive walk for each value of r across three meta-features: *inf.sd*, *sup.sd*, and *inc.sd*. The structure of this graph is similar to that of Figure 3. While the number of evaluations increases with the value of r , distinct configurations of r can lead the adaptive walk to attain seemingly different meta-feature values. So, we do not see a clear trend in the *GAP* values as a function of r . These results may occur because the adaptive walk selects a dominant neighbor at each step. The size of the neighborhood influences the existence of dominant neighbors, which can shorten the length of the walk and, consequently, affect the resulting meta-feature value.

6.3 Meta-learning results

Table 1 shows the average merit m^* achieved using the RF regression model for 30 sampling configurations. Gray cells indicate the configurations where $m^* < 1$, and the values in bold highlight the best merit identified. From the 30 configurations tested, 12 yielded $m^* < 1$, signifying that, in these cases, the RF regression model performed more effectively than the *SBS* in tackling the *ASP* for *MoWFLOP*. The configuration of $L = 50$ with $r = 0.25$ for the random walk, as well as $r = 1.0$ for the adaptive walk, achieved the best merit of $m^* = 0.96$. In the case of the adaptive walk, it was observed that as r increases, the value of m^* declines. Specifically, the adaptive walk produced $m^* < 1$ for $r > 0.05$ and $m^* = 1.0$ when $r = 0.05$. This trend occurs because a higher r leads the adaptive walk to explore a greater number of non-dominated neighbors, thereby enhancing the exploration of the search space and resulting in meta-features that are more suitable for prediction tasks. These findings demonstrate the benefits of employing meta-learning for automatically selecting algorithms for *MoWFLOP*.

The random walk configurations with $L = 50$ and $r = 0.25$ and the adaptive walk configuration at $r = 1.0$ obtained the best overall performance in terms of

Table 1: Average merit m^* for different sampling configurations

r	Random Walk					Adaptive Walk
	L = 100	L = 50	L = 25	L = 10	L = 5	
1.0	1.00	1.00	0.99	0.99	1.02	0.96
0.5	1.00	1.01	0.98	1.00	1.00	0.98
0.25	1.00	0.96	1.00	1.00	0.99	0.98
0.10	0.99	0.99	1.00	1.00	0.99	0.99
0.05	1.00	1.00	1.00	1.00	1.00	1.00

MSE, RMSE, MAE, MAPE, and R^2 , with the former outperforming the latter. These findings indicate a strong predictive capability of the RF model in these configurations. In contrast, the random walk configuration with $L = 25$ and $r = 0.05$ exhibited the poorest results across all metrics, including MSE, RMSE, MAE, and MAPE. These results can be attributed to the small neighborhood explored in this configuration, leading to poor search space sampling. Figure 5 highlights the ten most significant meta-features of the RF model for the top-performing random walk and adaptive walk configurations. The result shows that the importance of meta-features varies depending on the extraction method employed.

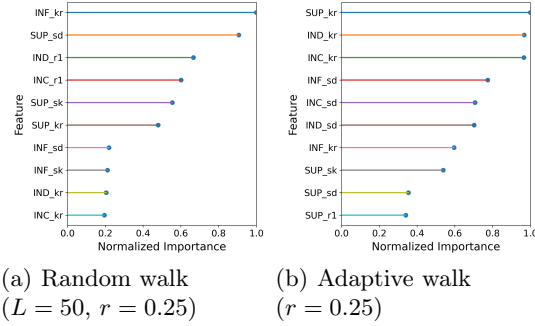


Fig. 5: The ten most significant meta-features

7 Conclusion and Future Work

This paper offered valuable insights into the use of Pareto-based meta-features for characterizing the search space of *MoWFLOP*. This problem represents a critical phase in the offshore wind farm design. Integrating meta-learning with Pareto-based meta-features, a machine learning model was developed to predict the performance of evolutionary algorithms (*NSGA-II* and *MOEA/D*) when applied to this problem.

Experiments indicated that *NSGA-II* and *MOEA/D* exhibited distinct performance profiles across 300 instances, with *MOEA/D* achieving superior re-

sults in most cases. Further, the analysis revealed that the length of the walk and the neighborhood size explored by random walk and adaptive walk methods incurred varying computational costs, thereby influencing the accuracy of the meta-features collected. Models relying on Pareto-based meta-features extracted by adaptive walk demonstrated enhanced performance, positively correlated with an increase in the number of neighbors explored.

This study has established significant foundations for future research applied to wind farm optimization problems, which could explore a broader range of meta-features and deal with more metaheuristics.

Acknowledgments. This research was supported by the Bahia State Research Support Foundation (FAPESB) through the PIBIC/UFBA and MOVE programs, CMCAD Fabíola Greve (UFBA), the Brazilian Federal Agency for Support and Evaluation of Graduate Education (CAPES), and the National Council for Scientific and Technological Development (CNPq), under grant number 402842/2023-5.

References

1. Adaramola, M., Krogstad, P.Å.: Experimental investigation of wake effects on wind turbine performance. *Renewable Energy* **36**(8), 2078–2086 (2011)
2. Beiter, P., Musial, W., Smith, A., Kilcher, L., Damiani, R., Maness, M., Sirnivas, S., Stehly, T., Gevorgian, V., Mooney, M., et al.: A spatial-economic cost-reduction pathway analysis for us offshore wind energy development from 2015–2030. Tech. rep., National Renewable Energy Lab.(NREL), Golden, CO (United States) (2016)
3. Biswas, P.P., Suganthan, P.N., Amaratunga, G.A.: Decomposition based multi-objective evolutionary algorithm for windfarm layout optimization. *Renewable energy* **115**, 326–337 (2018)
4. Brazdil, P., Rijn, J.N.v., Soares, C., Vanschoren, J.: *Metalearning: Applications to Automated Machine Learning and Data Mining*. Springer Science & Business Media (2022)
5. Breiman, L.: Random forests. *Machine learning* **45**, 5–32 (2001)
6. Cazzaro, D., Pisinger, D.: Variable neighborhood search for large offshore wind farm layout optimization. *Computers & Operations Research* **138**, 105588 (2022)
7. Cosson, R., Derbel, B., Liefvooghe, A., Aguirre, H., Tanaka, K., Zhang, Q.: Decomposition-based multi-objective landscape features and automated algorithm selection. In: *Evolutionary Computation in Combinatorial Optimization: 21st European Conference, EvoCOP 2021, Held as Part of EvoStar 2021, Virtual Event, April 7–9, 2021, Proceedings* 21. pp. 34–50. Springer (2021)
8. Cosson, R., Derbel, B., Liefvooghe, A., Verel, S., Aguirre, H., Zhang, Q., Tanaka, K.: Cost-vs-accuracy of sampling in multi-objective combinatorial exploratory landscape analysis. In: *Proceedings of the Genetic and Evolutionary Computation Conference*. pp. 493–501 (2022)
9. Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE transactions on evolutionary computation* **6**(2), 182–197 (2002)
10. Fernandes, I.F.C., Goldberg, E.F.G., Maia, S.M.D.M.: A novel multi-agent architecture based on decomposition and learning automata to hybridize multi-objective metaheuristics. *Memetic Computing* **17**(2), 1–28 (2025)

11. Fernandes, I.F.C., Silva, I.R.M., Goldberg, E.F.G., Maia, S.M.D.M., Goldberg, M.C.: A PSO-inspired architecture to hybridise multi-objective metaheuristics. *Memetic Computing* **12**(3), 235–249 (2020)
12. Fernandes, I.F.C.: Hybridizing metaheuristics for multi-and many-objective problems in a multi-agent architecture (2022)
13. Jensen, N.: A note on wind generator interaction. Technical report. Risø National Laboratory (1983)
14. Lee, J., Zhao, F.: Global Wind Report 2024. <https://www.gwec.net/reports/globalwindreport> (2024), accessed: 2025-03-14
15. Liefvooghe, A., Daolio, F., Verel, S., Derbel, B., Aguirre, H., Tanaka, K.: Landscape-aware performance prediction for evolutionary multiobjective optimization. *IEEE Transactions on Evolutionary Computation* **24**(6), 1063–1077 (2019)
16. Lindauer, M., van Rijn, J.N., Kotthoff, L.: The algorithm selection competitions 2015 and 2017. *Artificial Intelligence* **272**, 86–100 (2019)
17. Machado, M.O., Fernandes, I.F.C., Maia, S.M.D.M., Goldberg, E.F.G.: A hybrid multi-agent metaheuristic for the offshore wind farm cable routing problem. *Expert Systems with Applications* **255**, 124668 (2024)
18. Rice, J.R.: The algorithm selection problem. In: *Advances in computers*, vol. 15, pp. 65–118. Elsevier (1976)
19. Sorkhabi, S.Y.D., Romero, D.A., Beck, J.C., Amon, C.H.: Constrained multi-objective wind farm layout optimization: Novel constraint handling approach based on constraint programming. *Renewable energy* **126**, 341–353 (2018)
20. Yu, X., Lu, Y.: Reinforcement learning-based multi-objective differential evolution for wind farm layout optimization. *Energy* **284**, 129300 (2023)
21. Zhang, D., Liu, Z., Liang, Y., Li, C., Zhang, J., Hu, G.: Bilevel optimization of non-uniform offshore wind farm layout and cable routing for the refined lcoe using an enhanced PSO. *Ocean Engineering* **299**, 117244 (2024)
22. Zhang, Q., Li, H.: MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on evolutionary computation* **11**(6), 712–731 (2007)
23. Zheng, Y., Wang, J.S., Zhu, J.H., Zhang, X.Y., Xing, Y.X., Zhang, Y.H.: MORSA: Multi-objective reptile search algorithm based on elite non-dominated sorting and grid indexing mechanism for wind farm layout optimization problem. *Energy* **293**, 130771 (2024)
24. Zitzler, E., Thiele, L.: Multiobjective optimization using evolutionary algorithms—a comparative case study. In: *International conference on parallel problem solving from nature*. pp. 292–301. Springer (1998)
25. Zitzler, E., Thiele, L., Laumanns, M., Fonseca, C.M., Da Fonseca, V.G.: Performance assessment of multiobjective optimizers: An analysis and review. *IEEE Transactions on evolutionary computation* **7**(2), 117–132 (2003)
26. Zou, F., Chen, D., Liu, H., Cao, S., Ji, X., Zhang, Y.: A survey of fitness landscape analysis for optimization. *Neurocomputing* **503**, 129–139 (2022)